

Завдання №3: Реалізація бітового потоку

1 Мета роботи

Опанувати методи роботи із потоками бітів та із масивами даних на рівні окремих бітів.

2 Основне завдання

У даній частині роботи вам необхідно реалізувати інтерфейс для роботи із двійковими потоками даних. Рекомендується зробити його як надбудову до стандартного для вашої мови програмування типу даних або класу роботи із бінарними (не текстовими) файлами або потоками.

Необхідно реалізувати дві функції:

- `ReadBitSequence` — функція, яка вчитує із файла послідовність бітів вказаної довжини;
- `WriteBitSequence` — функція, яка записує у файл послідовність бітів вказаної довжини.

Послідовності бітів для читання/запису бажано брати з масиву байтів (один байт = вісім бітів). Для подальших задач вам також будуть корисні методи запису бітових послідовностей зі змінних та/або масивів інших цілочисельних типів даних, але необхідно пам'ятати, що основною одиницею пам'яті для файлів є саме байти, а порядок байтів у типах *word*, *integer* та *long* залежить від особливостей архітектури та мови програмування. За помовчанням вважайте, що усі цілочисельні дані зображуються як числа в системі числення з основою 256, а байти є окремими цифрами; відповідно, число типу *integer* зі значенням `1F2E3D4C` записується як масив байтів `[4C, 3D, 2E, 1F]`.

Після закриття файлу (потoku) він повинен автоматично вирівнюватись нульовими бітами до границі байту.

Приклад роботи. Нехай вам необхідно записати у файл дві послідовності з дев'яти бітів: `'100001111'` та `'011101110'`. Перший неочевидний момент, з яким ви стикнетесь: біти у послідовностях нумеруються зліва направо, а в цілочисельних типах даних, таких як *integer*, — зправа наліво. У нашому випадку обидві послідовності пронумеровані від 0 до 8, і бітом номер 1 у першій послідовності є '0'. Однак як масиви байтів наші послідовності будуть виглядати так:

```
a1 = [E1, 01]    // двійкові зображення: 11100001, 00000001
a2 = [EE, 00]    // двійкові зображення: 11101110, 00000000
```

Припустимо, що наш бітовий потік є на початку порожнім, і виконаємо послідовний запис наших послідовностей:

```
WriteBitSequence(BitStream, a1, 9),
WriteBitSequence(BitStream, a2, 9).
```

Після виконання даних операцій у потік буде записано ланцюг бітів `'100001111011101110'`, а у вигляді послідовності байтів він буде виглядати так:

```
E1 DD 01 // двійкові зображення: 11100001, 11011101 00000001
```

Зверніть увагу, що після запису першої послідовності ніякого вирівнювання під границю байтів не відбувається; друга послідовність записується саме в тому місці, де завершилась перша. Фінальне вирівнювання нульовими бітами відбувається тільки після закриття файлу.

Теперь, якщо відкрити файл наново та зробити два вчитування:

```
ReadBitSequence(BitStream, b1, 11),
```

```
ReadBitSequence(BitStream, b2, 7),
```

то масиви `b1` та `b2` будуть виглядати таким чином:

```
b1 = [E1, 05] // двійкові зображення: 11100001, 00000101
```

```
b2 = [3B] // двійкові зображення: 00111011
```

що відповідає двійковим послідовностям `'10000111101'` та `'1101110'` довжини 10 та 7 бітів відповідно. Зауважимо, що контроль за довжинами двійкових послідовностей залишається на користувачі, оскільки компілятори можуть оперувати тільки цілими байтами.

3 Вимоги до оформлення звіту

У даній тасці можете особливо не напружуватись: опишіть у звіті особливості вашої реалізації та приклад запису/читання декількох бітових послідовностей різних довжин.